SRS

Energy-Efficient HVAC Control

Occupancy-Aware HVAC Control Simulator

Team Members: Fanchao Yu, Minyuan Wang

October 6, 2026

Problem Statement

Fixed HVAC schedules may continue heating or cooling rooms when nobody is present. Small facilities need a clear way to explore how occupancy-aware control could reduce unnecessary operation. This project uses recorded office-room sensor data to demonstrate control decisions for a single office or factory administrative room.

Objective / Goal

Build a web application that replays temperature, humidity, and occupancy readings and selects simulated HVAC modes using configurable rules. Users can monitor conditions, override automatic decisions, review logs, and compare commanded heating/cooling time with a fixed-schedule baseline. The project evaluates software behavior; it does not claim measured energy savings or physically validated temperature control.

One Use Case (Primary)

User: Facility operator using the simulation dashboard.

1. The user loads the provided dataset and sets a comfort range, such as 20-24 degrees Celsius. These values are demonstration settings, not regulatory limits.

2. The user starts playback. The dashboard displays the current timestamp, temperature, humidity, occupancy, and simulated HVAC mode.

3. When the room is occupied, the controller requests heating below the lower limit, cooling above the upper limit, or standby within the configured range.

4. After the room remains unoccupied for the configured delay, the controller enters Eco mode and disables simulated heating/cooling.

5. The user may select a manual mode. The application records the change and its reason.

6. The user ends the session and reviews the control history and baseline comparison.

Scope of the Project

- One room and one simulated HVAC unit.
- Historical sensor-data import, validation, and chronological playback with start, pause, and reset controls.
- Dashboard for temperature, relative humidity, occupancy, and controller state.
- Rule-based heating, cooling, standby, and Eco modes with manual override.
- Configurable comfort range, vacancy delay, hysteresis, and minimum mode duration.
- Persistent settings, session history, and control-event logs.
- A fixed-schedule comparison using the same recorded inputs and thermostat logic.
- Desktop web application with a responsive layout.

Out of Scope: Physical HVAC integration, multi-room optimization, occupancy-model training, equipment safety certification, and claims of actual energy savings. Humidity is displayed but does not control a dehumidifier in the first release.

Tech Stack

Frontend:

- React with TypeScript
- Tailwind CSS for interface styling; Recharts for trend visualization

Backend:

- Python with FastAPI; REST API with JSON responses
- Pandas for parsing and validating sensor data

Database:

- SQLite for local settings, simulation sessions, and event history

Control Logic:

- Deterministic Python state machine; no machine learning required for the first release

Development Tools:

- Git and GitHub for version control
- Jira for tasks and Confluence for requirements and design documentation
- Pytest for backend tests; Vitest and React Testing Library for frontend tests
- GitHub Actions for automated checks

Hosting / Deployment:

- Local demonstration using the React frontend and FastAPI backend; a hosted deployment is optional

External Interface Requirements

User Interfaces

- Dashboard: playback controls, sensor values, mode indicator, and temperature/occupancy charts.
- Settings: comfort bounds, vacancy delay, mode-hold duration, hysteresis, and baseline schedule.
- History and Report: timestamped mode changes, manual actions, session parameters, and runtime comparison.

Hardware Interfaces

- A standard computer and web browser; no physical sensors or HVAC actuators are included.
- Sensor inputs are replayed from a dataset, and HVAC outputs are simulated commands. Instructor approval of this software-only scope remains to be confirmed.

Software Interfaces

- Data source: UCI Occupancy Detection dataset [1], containing 20,560 observations and timestamp, temperature, humidity, light, CO2, humidity ratio, and occupancy fields.
- Import: parse the supplied comma-delimited text files; combine and sort by timestamp, remove exact duplicates, and reject conflicting timestamps or invalid required fields.
- The first release uses recorded occupancy labels directly. Light and CO2 are not required for control.
- Frontend/backend interface: REST endpoints for playback, current state, settings, manual commands, logs, and reports.
- Database interface: store session configuration and events through the backend.

Functional Requirements

- **FR-01 - Data import:** Validate timestamps, numeric temperature, humidity between 0 and 100%, and occupancy values of 0 or 1. Invalid rows must be identified before playback.
- **FR-02 - Playback:** Process records chronologically. Pause must stop progression; reset must start a new session without deleting earlier logs.
- **FR-03 - Monitoring:** Display the current record, units, occupancy status, operating mode, and whether control is automatic or manual.
- **FR-04 - Automatic control:** Select heating, cooling, standby, or Eco from configured rules. Heating and cooling must never be active together.

Functional Requirements (continued)

- **FR-05 - Stable transitions:** Default comfort bounds are 20 and 24 degrees Celsius, with a 0.5-degree hysteresis and a five-minute minimum mode duration. Heating stops at 20.5 degrees; cooling stops at 23.5 degrees. All timing uses dataset time. Values are configurable demonstration assumptions.
- **FR-06 - Vacancy and overrides:** After ten continuous unoccupied minutes, enter Eco. Reoccupancy resumes thermostat control. Manual mode remains active until automatic mode is selected. Invalid-data handling takes precedence over all commands; manual and vacancy transitions bypass the normal mode-hold timer.
- **FR-07 - Settings:** Reject a lower bound greater than or equal to the upper bound, negative durations, and hysteresis greater than or equal to half the comfort-band width. Save valid changes and record old/new values.
- **FR-08 - Event history:** Save each mode change with session ID, timestamp, sensor values, previous/new mode, and reason. Events remain available after restarting the application.
- **FR-09 - Comparison:** Compare automatic occupancy-aware control with a configurable daily schedule (default 08:00-18:00) that uses the same thermostat logic but ignores occupancy. Exclude manual-override intervals from both totals.
- **FR-10 - Runtime calculation:** Sum heating/cooling command durations over valid consecutive record intervals. Exclude gaps longer than five minutes and show excluded time. Show percentage reduction as (baseline - proposed) / baseline x 100; report N/A when baseline runtime is zero.
- **FR-11 - Data gaps:** Invalid input or a gap exceeding five recorded minutes places the controller in Data Unavailable with no active command until valid input resumes.

Non-Functional Requirements

- **Performance:** During normal local playback, display a processed record within one second of receipt; verify with 100 sequential updates.
- **Correctness:** All defined control-transition and boundary test cases must pass before submission.
- **Usability:** Show units, mode labels, and control reasons in text; do not rely on color alone.
- **Reliability:** Replaying identical inputs and settings must produce identical decisions. Persist committed logs and settings across application restarts.
- **Transparency:** Label every page as a simulation. Recorded temperatures are fixed inputs and do not respond to commands. Runtime differences are not measured energy savings.

Estimated Timeline

Proposed eight-week schedule beginning after requirements approval; adjust to the course final deadline.

1. Planning and Design (Week 1)

Finalize the SRS, confirm simulation scope, create Jira tasks and a Confluence page, inspect data, and design page wireframes and database tables.

2. Core Development (Weeks 2-3)

Implement dataset import, playback, backend endpoints, dashboard, and persistent session storage.

3. Control and Reporting (Weeks 4-5)

Implement automatic rules, manual mode, settings validation, event logs, and baseline runtime comparison.

4. Testing and Quality Assurance (Weeks 6-7)

Test temperature boundaries, vacancy timers, hysteresis, overrides, data gaps, persistence, and runtime calculations. Complete integration and usability checks.

5. Delivery and Documentation (Week 8)

Prepare a reproducible local setup, user guide, API notes, final report, and demo presentation.

NIST Framework Alignment

The project uses NIST SP 1500-201, Framework for Cyber-Physical Systems: Volume 1, Overview [2], as a design reference. The following are project-specific applications of its aspects, not a certification or a prescribed NIST energy-saving formula:

- **Functional and human:** Define explicit controller states and provide understandable decisions and manual control.
- **Timing and data:** Use recorded timestamps, consistent units, input validation, and explicit missing-data behavior.
- **Trustworthiness:** Test deterministic transitions and retain decision logs so results can be checked.
- **Boundaries:** Clearly separate recorded physical measurements from simulated control outputs. Assess efficiency only through the stated runtime proxy.

References

[1] Candanedo, L. (2016). Occupancy Detection. UCI Machine Learning Repository. Dataset page: UCI Occupancy Detection.

[2] Griffor, E. R., Greer, C., Wollman, D. A., and Burns, M. J. (2017). Framework for Cyber-Physical Systems: Volume 1, Overview. NIST SP 1500-201. https://doi.org/10.6028/NIST.SP.1500-201.